

Proyecto 2: JSON Index

Portada

Tecnologico de Costa Rica

Escuela de Computacion

Principios de Sistemas Operativos

Segundo Proyecto

Tema: Libreria e interfaz de linea de comando para indexar archivos JSON grandes.

Introduccion

El proyecto implementa una alternativa sencilla a los archivos indexados tradicionales, aplicada a documentos JSON. La idea central es construir un archivo auxiliar con extension `.jnx` que almacena las rutas de los valores del JSON junto con las posiciones de byte donde esos valores se encuentran en el archivo original.

Con este indice, una busqueda no necesita parsear completamente el JSON cada vez. En su lugar, se aplican expresiones regulares sobre las rutas guardadas y se extraen directamente los fragmentos usando `fseek`.

Analisis del problema

Un archivo JSON grande puede tener muchos objetos, arreglos y valores escalares. Si cada consulta requiere leer y parsear todo el documento, el costo de busqueda crece con el tamaño completo del archivo. El problema se divide en dos etapas:

- Construir un indice de rutas absolutas.
- Resolver consultas posteriores usando ese indice.

El parser asume que el JSON esta bien formado, segun la especificacion del proyecto. No se utilizan bibliotecas externas para parsear JSON.

Solucion propuesta

La solucion se compone de tres partes:

- `libjsonindex.h`: interfaz publica de la libreria.
- `libjsonindex.c`: implementacion del parser, construccion del indice,

busqueda con `regex.h` y extraccion con `fseek`.

- `jqindex.cpp`: programa de linea de comando.

El parser recorre el archivo caracter por caracter. Cuando encuentra un objeto, un arreglo o un valor escalar, registra el camino actual y los bytes donde inicia y termina el valor. Las posiciones se calculan sobre el archivo original leido en modo binario para conservar los offsets reales.

Formato del indice JNX

El archivo `.jnx` es texto plano. Cada linea tiene el siguiente formato:

```
ruta<TAB>inicio<TAB>fin
```

Donde:

- `ruta` es el camino absoluto del valor. La raiz se guarda como `$` y los hijos se guardan como `/usuarios/0/nombre`.
- `inicio` es el byte donde empieza el valor en el JSON original.
- `fin` es el byte inmediatamente posterior al final del valor.

Se usa ``fin`` exclusivo porque permite calcular la longitud del fragmento como:

```
longitud = fin - inicio
```

Sintaxis de busqueda

Las busquedas usan expresiones regulares POSIX extendidas mediante ``regex.h``. El comando acepta expresiones con ``$`` como raiz:

```
./jqindex search datos.json "$/usuarios/[0-9]+/nombre"
```

Internamente, la expresion anterior se compara contra rutas como:

```
/usuarios/0/nombre  
/usuarios/1/nombre
```

La implementacion ancla la expresion completa para evitar coincidencias parciales inesperadas. Por ejemplo, ``$/usuarios/[0-9]+/nombre`` coincide con ``/usuarios/0/nombre``, pero no con ``/usuarios/0/nombre/apellido``.

Ejecucion de pruebas

Para compilar:

```
make
```

Para generar el indice del archivo de ejemplo:

```
./jqindex build examples/datos.json examples/datos.jnx
```

Para ejecutar busquedas:

```
./jqindex search examples/datos.json "$/usuarios/[0-9]+/nombre" examples/datos.jnx  
./jqindex search examples/datos.json "$./activo" examples/datos.jnx  
./jqindex search examples/datos.json "$/config/opciones/modo" examples/datos.jnx
```

Para ejecutar las pruebas automatizadas:

```
make test
```

Analisis de resultados

Las pruebas automatizadas verifican cuatro casos principales:

- Recuperacion de multiples cadenas: ``["Ana", "Luis"]``.
- Recuperacion de multiples booleanos: ``[true, false]``.
- Recuperacion de un unico escalar: ``"auto"``.
- Busqueda sin coincidencias: ``[]``.

Los resultados muestran que el indice permite localizar fragmentos exactos del archivo original sin volver a parsear todo el documento durante la busqueda. La salida conserva los valores JSON originales, incluyendo comillas en cadenas, objetos, arreglos, numeros, booleanos y ``null``.

Limitaciones

El parser fue construido como una solución práctica para archivos JSON bien formados. Reconoce objetos, arreglos, cadenas, números, booleanos y ``null``. También maneja escapes comunes dentro de cadenas. Para componentes de ruta con ``/`` o ``~``, se aplica el escape compatible con JSON Pointer: ``/`` se convierte en ``~1`` y ``~`` se convierte en ``~0``.

La librería no pretende ser un parser JSON completo de producción; su objetivo es cumplir el alcance solicitado por el proyecto y demostrar el mecanismo de indexación.

Conclusiones

El proyecto demuestra que es posible separar el costo de parseo en una fase de construcción de índice y acelerar las consultas posteriores usando offsets de byte. El archivo ``.jnx`` es simple, legible y fácil de depurar.

La combinación de rutas absolutas, expresiones regulares y extracción directa con ``fseek`` permite recuperar valores individuales o conjuntos de valores sin procesar nuevamente todo el JSON. Esta estrategia conserva la idea de los archivos indexados tradicionales y la adapta a datos semiestructurados modernos.